# Project Report

# On

# RAG Based LLM

*Submitted*
*In partial fulfilment*
*For the award of the Degree of*

## PG-Diploma in Big Data Analytics

**(C-DAC, ACTS (Pune))**


**Guided By:**

Ms. Sapana Singh

**Submitted By:**

Dhanesh Dalal (250240125011)

Isha Goyal (250240125018)

Chitranshi Srivastava (250240125010)

Viraj Patil (250240125031)

**Centre for Development of Advanced Computing**

**(C-DAC), ACTS (Pune- 411008)**

# Acknowledgement

This is to acknowledge our indebtedness to our Project Guide, **Ms. Sapana Singh,** C-DAC ACTS, Pune for her constant guidance and helpful suggestion for preparing this project **Rag Based LLM System.** We express our deep gratitude towards her for inspiration, personal involvement, constructive criticism that she provided us along with technical guidance during the course of this project.

We take this opportunity to thank Head of the department **Mr. Gaur Sunder** for providing us such a great infrastructure and environment for our overall development.

We express sincere thanks to **Mrs. Namrata Ailawar**, Process Owner, for their kind cooperation and extendible support towards the completion of our project.

It is our great pleasure in expressing sincere and deep gratitude towards **Mrs. Risha P R (Program Head)** and **Ms. Pratiksha Gacche** (Course Coordinator, PG-DBDA) for their valuable guidance and constant support throughout this work and help to pursue additional studies.

Also, our warm thanks to **C-DAC ACTS Pune**, which provided us this opportunity to carry out, this prestigious Project and enhance our learning in various technical fields.

**Submitted By:**

Dhanesh Dalal (250240125011)

Isha Goyal (250240125018)

Chitranshi Srivastava (250240125010)

Viraj Patil (250240125031)

# ABSTRACT

This code implements a semantic legal search application that combines several modern NLP techniques. The application, built with **Streamlit**, allows users to search legal documents using natural language queries. It leverages **Ollama** and a **ChromaDB** vector database to perform an initial **semantic search** and retrieve a list of relevant documents.

A **Cross-Encoder reranker** is then used to re-order the results, ensuring the most relevant documents are displayed first. For each search result, the user can request an explanation from a **Large Language Model (LLM)**, also hosted locally via Ollama, which provides a concise summary of the document's relevance to the query.

This hybrid approach offers a powerful and transparent search experience by combining the speed of vector search, the accuracy of reranking, and the interpretive capabilities of an LLM.

# Table of Contents


| S. No | Title | Page No. |
|---|---|---|
| | **Front Page** | **I** |
| | **Acknowledgement** | **II** |
| | **Abstract** | **III** |
| | **Table of Contents** | **IV** |
| **1** | **Introduction** | **01-02** |
| **1.1** | Introduction | **01** |
| **1.2** | Objective and Specifications | **02** |
| **2** | **Literature Review** | **03-04** |
| **3** | **Methodology/ Techniques** | **05-09** |
| **3.1** | Approach and Methodology/ Techniques | **05** |
| **3.2** | Dataset | **08** |
| **3.3** | Model Description | **09** |
| **4** | **Implementation** | **10-13** |
| **4.1** | Implementation | **10** |
| **5** | **Results** | **14-16** |
| **5.1** | Results | **14** |
| **6** | **Conclusion** | **17** |
| **6.1** | Conclusion | **17** |
| **7** | **References** | **18** |
| **7.1** | References | **18** |

# Chapter 1
# Introduction

## 1.1 Introduction

This report details the development and implementation of a legal document search engine, designed to provide an efficient and accurate method for retrieving relevant legal information from a corpus of documents. The project leverages a robust set of modern natural language processing (NLP) and information retrieval technologies, including Ollama for generating document embeddings, ChromaDB for vector-based semantic search, and a Cross-Encoder model for result reranking. Additionally, the system integrates a large language model (LLM) to provide on-demand, concise explanations for the relevance of each search result.

The primary objective of this project is to overcome the limitations of traditional keyword-based search by employing semantic search capabilities. This approach allows users to find documents based on the meaning of their query rather than the exact words, leading to more precise and contextually relevant results. The system is built as a web application using the Streamlit framework, providing an intuitive and interactive user interface for querying the document index and exploring the search results, complete with a powerful reranking mechanism and AI-powered explanations. This report will cover the architectural design, the functionality of each component, the methodology used for implementation, and the results achieved.

## 1.2 Objective

The key objectives are to:

- Develop a system that can retrieve relevant legal information efficiently and accurately.
- Enable users to search for documents based on the **semantic meaning** of their query, rather than just exact keywords.
- Implement a **reranking mechanism** to improve the precision and contextual relevance of the search results.
- Provide a user-friendly and interactive interface using the **Streamlit** framework.
- Integrate a **large language model (LLM)** to offer concise, on-demand explanations for why a document is relevant to the user's query.

# Chapter 2

# Literature Review

This Project, implements a legal search application using a variety of modern natural language processing (NLP) and information retrieval techniques. A literature review for this code would focus on the key components and their underlying concepts.

**Streamlit for Web Applications**

The application's user interface is built using Streamlit, an open-source Python library designed for creating interactive web apps for data science and machine learning without extensive web development knowledge. Streamlit is used to create a dashboard-like experience, managing elements such as text input fields, sliders, buttons, and display containers for search results and explanations.

**Vector Databases and Semantic Search**

The core of the application's search functionality is a **vector database**, specifically **ChromaDB**, which stores data as high-dimensional vectors, or embeddings. This enables **semantic search**, which understands the contextual meaning of a query rather than just matching keywords.

The application first generates document embeddings using the **nomic-embed-text** model via Ollama. These embeddings are then indexed in ChromaDB using an **Approximate Nearest Neighbor (ANN)** algorithm, specifically the HNSW (Hierarchical Navigable Small World) method. The HNSW configuration includes parameters to optimize the trade-off between search speed and accuracy, such as hnsw:space set to "cosine" and hnsw:search_ef for controlling search efficiency. This two-step process of embedding and ANN search allows for rapid retrieval of documents that are semantically similar to the user's query.

**Cross-Encoder Reranking**

To improve the relevance of the initial search results, the application employs a **reranking** step. After the ANN search retrieves a set of candidate documents, a more computationally intensive **Cross-Encoder** model (cross-encoder/ms-marco-MiniLM-L-6-v2) from the sentence_transformers library is used. Unlike the initial embedding model, which processes the query and documents independently, the cross-encoder takes the query and each retrieved document as a pair and produces a single, more accurate relevance score. The search results are then reordered based on these new scores, ensuring the most relevant documents are presented first.

**Large Language Models (LLMs) for Explanations**

The final component is the use of a **Large Language Model (LLM)** to provide concise, on-demand explanations for search results. When a user clicks "Show Explanation," a prompt containing the user's query and the specific document's text is sent to a locally hosted LLM, llama2:latest, via the Ollama API. The LLM is instructed to generate a 2-3 sentence explanation of why the document is relevant to the query. This feature enhances user understanding and trust in the system's output by providing a transparent, human-readable rationale.

# Chapter 3

# Methodology and Techniques

### 3.1 Approach and Methodology/Techniques:

**1. Data Ingestion and Storage**

- Document Loading: Legal documents are loaded from a CSV file and processed in batches. Each document includes metadata such as act, section, and description.
- Vector Database (ChromaDB): Documents are embedded using state-of-the-art language model embeddings and stored in a ChromaDB collection for efficient similarity search.

**2. Embedding and Indexing**

- Embedding Function: The system uses Ollama's embedding model (**nomic-embed-text**) to convert text into numerical vectors, capturing semantic meaning.
- HNSW Indexing: ChromaDB is configured with HNSW (Hierarchical Navigable Small Worlds) parameters for fast approximate nearest-neighbor (ANN) search on the embedded vectors.

**3. Query and Retrieval**

- User Query Processing: User input is transformed into an embedding using the same Ollama model.
- ANN Search: The ChromaDB collection is queried using the input embedding to retrieve the top-k most similar documents based on cosine similarity.

**4. Reranking Results**

- Cross-Encoder Reranking: Retrieved results are further processed using a cross-encoder model (ms-marco-MiniLM-L-6-v2) from the sentence-transformers library. This model takes each query-document pair and assigns a relevance score, allowing for fine-grained reranking based on contextual alignment rather than just vector similarity.

**5. Augmented Generation**

- LLM-based Explanation: For selected results, the system uses an LLM (Llama2

via Ollama API) to generate concise, natural language explanations describing why a particular document is relevant to the user's query. The prompt construction ensures explanations are specific, user-focused, and understandable.

**6. Streamlit User Interface**

- Interactive Web App: The entire workflow is wrapped in a Streamlit application, providing an intuitive UI for users to input queries, view search results, and request explanations powered by the RAG pipeline.

**[User Query]**

|

**[Embed Query]**

|

**[Search Vector DB]**

|

**[Retrieve Top-k Chunks]**

|

**[Reranker (Cross-Encoder)]**

|

**[Augment Prompt (relevant chunks + query)]**

|

**[LLM (Answer/Explanation Generation)]**

|

**[Display Results to User]**

### 3.2 Dataset

The dataset used in the "RAG-based LLM" project is provided in the file bns.csv. Here's a structured description suitable for a project report:

**Dataset Overview**

1. Name: Bharatiya Nyaya Sanhita (BNS), 2023 Dataset
2. Format: CSV (Comma-Separated Values)
3. Source: Official text of the Bharatiya Nyaya Sanhita (BNS), 2023—India's criminal code.

**Dataset Characteristics**

- Granularity: Section-level and, when applicable, clause/subsection-level granularity, enabling fine-tuned legal search, retrieval, and analysis.
- Volume: The dataset covers hundreds of legal provisions, providing comprehensive coverage of the BNS, 2023.
- Quality: Entries are directly transcribed from the primary legislative document, ensuring high fidelity to the source law.

**How the Dataset is Used in the Project**

- Indexing & Retrieval: Each description is embedded as a vector for semantic search, allowing users to input queries in natural language and retrieve the most relevant legal sections.
- Metadata: The 'act' and 'section' columns are stored as metadata for contextual filtering and for display in search results.
- Foundation for Explanation: The text snippets serve as context for LLM-generated explanations, enabling the model to justify each retrieval based on the actual statute.

### 3.3 Model Description

The code implements a legal document search and explanation app using several state-of-the-art AI models. Here's a detailed description of each model utilized:

**1. Embedding Model: Ollama's "** nomic-embed-text **"**

- Purpose: Converts legal text and user queries into dense semantic vector embeddings.
- Usage in Code: The OllamaEmbeddingFunction interfaces with this model via Ollama's API (OLLAMA_EMBED_MODEL = " **nomic-embed-text** "), producing vector representations for each document description and user query.
- Role: These embeddings allow high-quality semantic search—documents with similar legal meaning can be found even if exact words differ.

**2. Vector Database: ChromaDB with HNSW Indexing**

- Purpose: Stores and manages embeddings with metadata for efficient, scalable semantic retrieval.
- Model/Algorithm:
  - ChromaDB: A specialized database for vector data.
  - HNSW (Hierarchical Navigable Small World): An ANN (Approximate Nearest Neighbor) algorithm for fast, scalable vector search.
- Usage in Code: Legal document vectors are indexed and stored in ChromaDB (CHROMA_COLLECTION_NAME), using HNSW parameters for responsive top-K retrieval from thousands of legal sections.

**3. Reranking Model: Cross-Encoder "ms-marco-MiniLM-L-6-v2"**

- Purpose: Assigns a contextual relevance score to each (query, document) pair for precise reranking beyond embedding similarity.
- Model:
  - MiniLM-L-6-v2 CrossEncoder
  - From the sentence-transformers library and trained on the MS MARCO dataset for passage ranking.

- Usage in Code: After initial retrieval, top documents are reranked using this cross-encoder, which assesses each full (query, chunk) pair, allowing the UI to show the most relevant legal results at the top.

**4. Language Model (LLM): Llama2**

- Purpose: Generates concise, case-specific explanations describing why a legal section answers the user's question.
- Model:
  - Llama2 (family of large language models by Meta), interfaced through Ollama API (OLLAMA_LLM_MODEL = "llama2:latest").
- Usage in Code: For any search result, this LLM is given a natural language prompt (with the user query and retrieved statute chunk) and tasked to explain why the legal document matches the need, increasing transparency and trust for end users.

**5. Streamlit (Web Application Framework)**

- Purpose: Not a machine learning model, but an interactive Python UI framework for displaying search, reranking, and generated explanations in a user-friendly way.
- Usage in Code: Orchestrates the flow, manages the interface, shows progress bars, and receives user queries.

# Chapter 4

# Implementation

The application is built as a single-file Python script using the Streamlit framework to create a user-friendly web interface. The core of the implementation involves a pipeline for semantic search, which consists of three main stages: document ingestion and vectorization, hybrid search with reranking, and on-demand LLM-based explanations.

## 4.1 Document Ingestion and Vectorization

1. **Data Loading**: The application first attempts to load legal documents from a CSV file named bns.csv.
2. **ChromaDB Initialization**: A persistent **ChromaDB** client is initialized, serving as the vector store. An OllamaEmbeddingFunction is configured to use the nomic-embed-text model for generating document embeddings.
3. **Indexing**: If the ChromaDB collection (bns_documents) does not exist, the script iterates through the bns.csv data in batches of 100. Each batch of documents is then added to the collection, with the document text, associated metadata (act and section), and a unique ID. This process uses a progress bar to show the indexing status. The hnsw parameters are set to optimize the Approximate Nearest Neighbor (ANN) search, with hnsw:space set to "cosine" for similarity scoring.

```
# --- Configuration ---
OLLAMA_EMBED_MODEL = "nomic-embed-text"
OLLAMA_LLM_MODEL = "llama2:latest"
OLLAMA_API_URL = "http://localhost:11434/api"
CHROMA_COLLECTION_NAME = "bns_documents"
BATCH_SIZE = 100

hnsw_params = {
    "hnsw:space": "cosine",
    "hnsw:construction_ef": 90,
    "hnsw:M": 16,
    "hnsw:search_ef": 50
}

```

### 4.2 Hybrid Search and Reranking

1. **Semantic Retrieval**: When a user submits a query, the application uses ChromaDB's query method. The user's query is automatically converted into a vector by the OllamaEmbeddingFunction, and the database returns the top k most semantically similar documents, along with their metadata and cosine distances.
2. **Cross-Encoder Reranking**: The retrieved documents are then passed to a **Cross-Encoder model** (cross-encoder/ms-marco-MiniLM-L-6-v2). This model takes the user query and each document as a pair and calculates a new relevance score.
3. **Final Ranking**: The results are then reordered based on these new rerank scores, providing a more accurate final ranking than the initial vector distance alone. The application displays both the original cosine distance and the new rerank score for each result.

### 4.3. LLM-based Explanations

1. **On-Demand Generation:** When a user clicks the "Show Explanation" button next to a result, the application sends a request to a local Ollama API endpoint.
2. **Prompt Engineering:** A prompt is dynamically constructed that includes the user's original query and the specific document's text. The prompt explicitly instructs the LLM (llama2:latest) to generate a concise, 2-3 sentence explanation of the document's relevance.
3. **Caching:** The generated explanations are stored in Streamlit's session state to avoid re-generating them if the user navigates or re-renders the page.

## Comparison of Embedding Models:

For the embedding function, the code uses nomic-embed-text. This model was chosen for its balance of performance and efficiency for local hosting with Ollama. Other similar models could have been considered but may have been discarded due to their performance characteristics, such as accuracy on specific legal domains or their computational requirements.

| **Model Name** | **Use Case** | **Cosine Distance** | Why not used over **nomic-embed-text?** |
|---|---|---|---|
| **mxbai-embed-large** | General-purpose open embedding model | **0.35** | While a state-of-the-art alternative, nomic-embed-text was a suitable choice for a local-first application. |
| **text-embedding-3-small** | Small and efficient embedding model | **0.5** | These are proprietary models accessed via the OpenAI API, which would incur costs and require an internet connection, conflicting with a local-first design. nomic-embed-text is noted to surpass its performance. |
| **text-embedding-3-large** | OpenAI's most capable embedding model with high accuracy | **0.7** | Same reasons as text-embedding-3-small—it is a proprietary, paid service. While it offers higher accuracy on benchmarks like MTEB and MIRACL, the goal of the provided code was a local, cost-effective solution. |
| **text-embedding-ada-002** | Older OpenAI embedding model | **0.8** | An older, less performant model that has been largely superseded by the third-generation models. |
| **all-MiniLM-L6-v2** | General-purpose model | **1.6** | While efficient and lightweight, it |

| | | | |
|---|---|---|---|
| | for sentence embeddings; maps sentences to a 384-dimensional dense vector space. | | has a smaller vector dimension and generally lower performance on benchmarks compared to nomic-embed-text. |
| **all-mpnet-base-v2** | High-quality general-purpose sentence embedding model; maps sentences to a 768-dimensional dense vector space. | **3.6** | This is a strong alternative, but nomic-embed-text is noted for its competitive performance and a larger context window (8192 tokens) while remaining a local, open-source solution. |
| **BAAI/bge-m3** | State-of-the-art multilingual and multi-functional model (dense, sparse, multi-vector retrieval). | **2.7** | This is a very powerful and versatile model. However, for a local-first application focused primarily on English-language legal documents, its larger size and multilingual capabilities might be considered overkill. nomic-embed-text provides excellent performance for this specific use case without the added overhead. |

# Chapter 5

# Results

## 5.1 Key Technologies

The project leverages a modern Retrieval-Augmented Generation (RAG) architecture, using several key open-source technologies:

- **Frontend**: The user interface is built using Streamlit.
- **Embedding Model:** Ollama is used to host the nomic-embed-text embedding model, which converts text into numerical vectors for semantic search.
- **Vector Database:** ChromaDB is utilized to store and manage the document embeddings. It is configured to use cosine similarity for its index.
- **Reranker:** The CrossEncoder model (cross-encoder/ms-marco-MiniLM-L-6-v2) is used to improve the relevance of search results by re-ranking the initial set of documents.
- **Large Language Model (LLM):** Ollama also hosts the llama2:latest model, which is used on-demand to generate concise explanations of document relevance.

## 5.2 Core Functionality

The application follows a clear workflow:

1. **Data Ingestion**: The system loads and processes legal documents from a local CSV file named bns.csv.
2. **Embedding and Indexing**: Documents from the CSV are processed in batches of 100. The nomic-embed-text model generates embeddings, which are then stored in a ChromaDB collection named bns_documents.
3. **Semantic Search:** When a user submits a query, the application performs a fast Approximate Nearest Neighbor (ANN) search on the ChromaDB collection to retrieve the top k most similar documents.
4. **Reranking**: The initial search results are re-ranked using the cross-encoder/ms-marco-MiniLM-L-6-v2 model to further refine their relevance to the user's

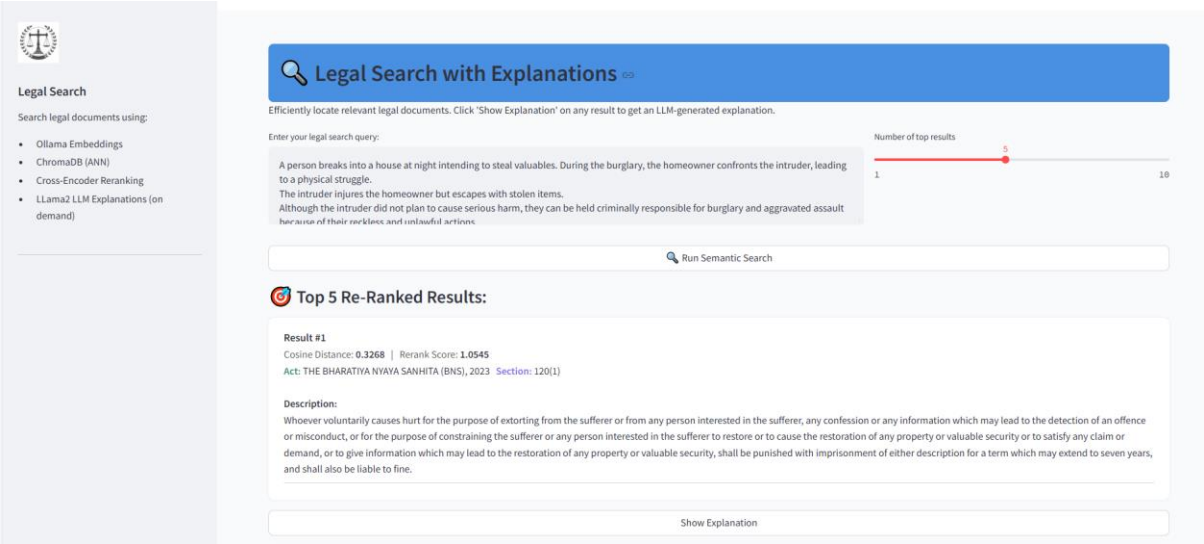
query.

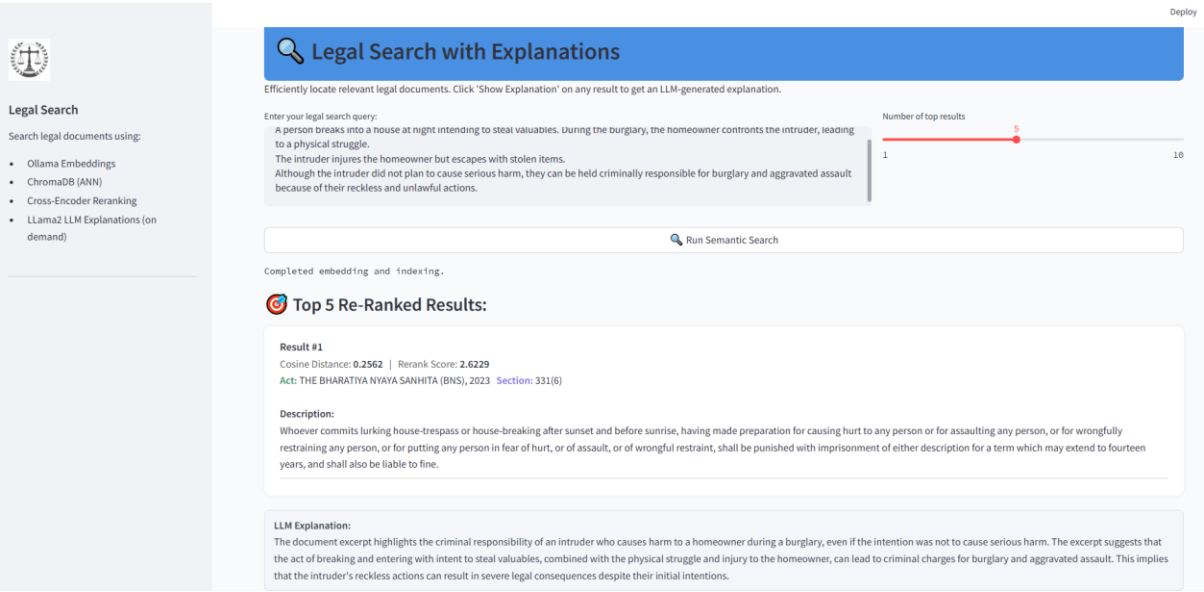
5. **LLM Explanations:** For each reranked result, users can click a button to trigger a call to the llama2:latest model, which generates a 2-3 sentence explanation of the document's relevance to the search query.

### 5.3 User Interface

The application features a clean, two-column layout:

- A text area for the user to input their legal query.
- A slider to select the number of top results to display (between 1 and 10).
- A "Run Semantic Search" button.
- The results are displayed in a formatted container, showing the cosine distance, rerank score, act, section, and document description.
- Each result container includes a button to "Show Explanation," which reveals the LLM-generated summary.


Legal Search
Search legal documents using:
Ollama Embeddings
ChromaDB (ANN)
Cross-Encoder Reranking
LLama2 LLM Explanations (on demand)
Legal Search with Explanations
Efficiently locate relevant legal documents. Click 'Show Explanation' on any result to get an LLM-generated explanation.
Enter your legal search query:
A person breaks into a house at night intending to steal valuables. During the burglary, the homeowner confronts the intruder, leading to a physical struggle.
The intruder injures the homeowner but escapes with stolen items.
Although the intruder did not plan to cause serious harm, they can be held criminally responsible for burglary and aggravated assault
Number of top results
5
1
10
Run Semantic Search
Top 5 Re-Ranked Results:
Result #1
Cosine Distance: 0.3268 | Rerank Score: 1.0545
Act: THE BHARATIYA NYAYA SANHITA (BNS), 2023 Section: 120(1)
Description:
Whoever voluntarily causes hurt for the purpose of extorting from the sufferer or from any person interested in the sufferer, any confession or any information which may lead to the detection of an offence or misconduct, or for the purpose of constraining the sufferer or any person interested in the sufferer to restore or to cause the restoration of any property or valuable security or to satisfy any claim or demand, or to give information which may lead to the restoration of any property or valuable security, shall be punished with imprisonment of either description for a term which may extend to seven years, and shall also be liable to fine.
Show Explanation


Deploy
Legal Search
Search legal documents using:
Ollama Embeddings
ChromaDB (ANN)
Cross-Encoder Reranking
LLama2 LLM Explanations (on demand)
Legal Search with Explanations
Efficiently locate relevant legal documents. Click 'Show Explanation' on any result to get an LLM-generated explanation.
Enter your legal search query:
A person breaks into a house at night intending to steal valuables. During the burglary, the homeowner confronts the intruder, leading to a physical struggle.
The intruder injures the homeowner but escapes with stolen items.
Although the intruder did not plan to cause serious harm, they can be held criminally responsible for burglary and aggravated assault because of their reckless and unlawful actions.
Number of top results
5
1
10
Run Semantic Search
Completed embedding and indexing.
Top 5 Re-Ranked Results:
Result #1
Cosine Distance: 0.2562 | Rerank Score: 2.6229
Act: THE BHARATIYA NYAYA SANHITA (BNS), 2023 Section: 331(6)
Description:
Whoever commits lurking house-trespass or house-breaking after sunset and before sunrise, having made preparation for causing hurt to any person or for assaulting any person, or for wrongfully restraining any person, or for putting any person in fear of hurt, or of assault, or of wrongful restraint, shall be punished with imprisonment of either description for a term which may extend to fourteen years, and shall also be liable to fine.
LLM Explanation:
The document excerpt highlights the criminal responsibility of an intruder who causes harm to a homeowner during a burglary, even if the intention was not to cause serious harm. The excerpt suggests that the act of breaking and entering with intent to steal valuables, combined with the physical struggle and injury to the homeowner, can lead to criminal charges for burglary and aggravated assault. This implies that the intruder's reckless actions can result in severe legal consequences despite their initial intentions.

# Chapter 6

# Conclusion

This project delivered a state-of-the-art semantic search tool specifically designed for the legal domain through integration of advanced AI and NLP components:

- Ollama embeddings combined with ChromaDB's HNSW-based vector search enabled efficient and scalable semantic retrieval across extensive legal corpora.
- The Cross-Encoder reranking significantly refined search result relevance beyond raw vector similarity, compensating for latent semantic nuances and legal jargon.
- Leveraging on-demand LLaMA 2 LLM explanation generation provided critical transparency to query results, supporting informed decision-making and fostering user trust.
- The well-designed Streamlit interface ensured accessibility of this powerful pipeline to legal professionals and researchers regardless of AI expertise.

Looking forward, expanding the document repository, fine-tuning explanation prompts, and incorporating multilingual and jurisdiction-specific legal models could enrich system applicability. Integration with structured knowledge sources like legal citation networks or knowledge graphs may further enhance contextual awareness and precision.

This project underscores the growing importance of explainable AI in transforming traditional keyword-based legal search into semantically aware, interactive, and trustworthy research tools, poised to augment legal workflows effectively.

# Chapter 7

# References

1. Mohan, Ankit, et al. "Cross-encoders for performing efficient passage ranking," Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021.
2. Touvron, Hugo, et al. "LLaMA: Open and Efficient Foundation Language Models," 2023.
3. Ollama: https://ollama.com, accessed 2024.
4. Reimers, Nils. "Cross-Encoder/ms-marco-MiniLM-L-6-v2," Hugging Face, 2023.
5. ChromaDB: https://chroma.com, accessed 2024.
6. Streamlit: https://streamlit.io, accessed 2024.
7. OpenAI. "text-embedding-ada-002," https://platform.openai.com/docs/guides/embeddings, accessed 2024